Contents

The Tabular ML Handbook **1**
Foundation models and gradient-boosted trees, end to end, on real heavy datasets 1
Table of contents 1
Part 0 — Why This Repository Exists 2
Part 1 — Google TabFM 2
1.1 What TabFM Is 2
1.2 Environment and a Real Bug We Hit 3
1.3 Classification: UCI Adult Income 3
1.4 Regression: California Housing 4
Part 2 — The Regression Model Zoo (NYC Green Taxi) 5
2.1 The Dataset 5
2.2 TabICLv2 5
2.3 CatBoost 6
2.4 LightGBM 6
2.5 Head-to-Head 6
Part 3 — The Classification Model Zoo (Forest Covertype) 7
3.1 The Dataset 7
3.2 TabPFN v3 7
3.3 Mitra 8
3.4 XGBoost 9
3.5 Head-to-Head 10
Part 4 — Cross-Cutting Lessons 10
References 11

The Tabular ML Handbook

Foundation models and gradient-boosted trees, end to end, on real heavy datasets

This is the long-form companion to this repository's `README.md`. Where the README gives you the headline numbers, this document walks through *why* each model works the way it does, *how* each notebook is built, and *what* the code is actually doing at every step — theory, definitions, full code, and results together, for all eight notebooks.

If you want the short version with just the comparison tables, read `COMPARISON.md` instead. If you want to run any of this yourself, every code block below is copied verbatim from an executed notebook in `notebooks/`.

Table of contents

- Part 0 — Why This Repository Exists
- Part 1 — Google TabFM
- Part 2 — The Regression Model Zoo (NYC Green Taxi)
  - 2.1 The Dataset
  - 2.2 TabICLv2
  - 2.3 CatBoost
  - 2.4 LightGBM
  - 2.5 Head-to-Head
- Part 3 — The Classification Model Zoo (Forest Covertype)
  - 3.1 The Dataset
  - 3.2 TabPFN v3
  - 3.3 Mitra
  - 3.4 XGBoost
  - 3.5 Head-to-Head
- Part 4 — Cross-Cutting Lessons

- References

Part 0 — Why This Repository Exists

For most of the last decade, the answer to "what model should I use for tabular data?" was settled: a well-tuned gradient-boosted tree (XGBoost, LightGBM, CatBoost). Deep learning dominated text and vision, but tabular data resisted the same transfer-learning playbook for a structural reason — **a spreadsheet has no shared vocabulary.** A column called `age` in one dataset and `hours-per-week` in another don't share a token embedding the way "dog" means the same thing across English sentences. Column semantics, count, order, and scale differ across every dataset, so a model pretrained on one schema has no obvious way to transfer to a completely different one.

In-context learning (ICL) is the trick that changed this. Instead of building a shared vocabulary across arbitrary schemas, an ICL tabular model treats your *entire labeled training set* as a prompt: it reads your training rows as context, then predicts new rows in a single forward pass — no gradient updates, no per-dataset training. The model is pretrained once, on a huge number of *synthetic* tables, to learn the general skill of "infer patterns from a table of examples," and that skill transfers to any new table because every dataset supplies its own context.

This lineage started with **TabPFN** (2023), continued through **TabICL** (which scaled ICL to larger tables via row compression), and now includes **Google's TabFM** and AutoGluon's **Mitra** — four different organizations converging on variations of the same core idea within about two years of each other.

This repository asks a direct, practical question about all of them: **does any of this actually beat a properly tuned classical GBM, on a genuinely large, messy, real dataset, once you account for the compute and tuning effort each approach actually costs?** The honest answer, worked out across eight notebooks, is "it depends on scale and how much tuning effort you're willing to spend" — not a simple yes or no in either direction. Part 4 below has the full synthesis; the rest of this document is the model-by-model detail that gets you there.

Part 1 — Google TabFM

Notebooks: `google_tabfm_classification_tutorial.ipynb`, `google_tabfm_regression_tutorial.ipynb`

1.1 What TabFM Is

TabFM was announced by **Google Research** (not DeepMind, not a Cloud product team) on June 30, 2026, via a research blog post, an open GitHub repo, and Hugging Face model cards. It is explicitly positioned as the tabular sibling to Google's own time-series foundation model, **TimesFM** — same "one pretrained model, many downstream datasets" idea, applied to tables.

Architecture — a three-stage hybrid of two existing ICL lineages:

1. **Column attention** (TabPFN/Set-Transformer style): each cell is embedded via Fourier features, then columns attend to each other via induced self-attention.
2. **Row compression** (TabICL style): CLS tokens with Rotary Position Embedding compress every full row into one dense vector — this is what keeps the next stage tractable on large tables.
3. **In-context Transformer**: a 24-block causal transformer runs over the compressed row vectors, treating training rows as context and test rows as the query.

Hyperparameter	Value
Embedding dim	256
Column-attention blocks	3 (4 heads, 256 induced points)

Hyperparameter	Value
Row-attention blocks	3 (8 heads, 8 CLS tokens)
ICL transformer blocks	24 (8 heads)
Max classes	**10 (hard limit)**
Optimized table width	up to ~500 features

Training data is **entirely synthetic** — hundreds of millions of tables generated from structural causal models, chosen specifically to avoid the scarcity and privacy problems of real tabular data at scale. As of this writing there is no arXiv paper or technical report — the public record is the blog post, GitHub README, and Hugging Face model cards.

License split, worth knowing before you build anything on this: the *code* is Apache-2.0, but the pretrained *weights* ship under a custom **"TabFM Non-Commercial License v1.0."**

1.2 Environment and a Real Bug We Hit

`pip install tabfm[pytorch]` from PyPI, as documented in TabFM's own Quick Start, ships a loader that hardcodes the filename `pytorch_model.bin` — but the actual Hugging Face weights are stored as `model.safetensors`. This raises a bare `FileNotFoundError` out of the box. The fix, which the notebooks use directly:

```
uv pip install --python {sys.executable} -q \
    "tabfm[pytorch] @ git+https://github.com/google-research/tabfm.git" \
    safetensors scikit-learn pandas numpy matplotlib seaborn xgboost
```

Installing from GitHub `main` uses the corrected Hugging Face loading path. `safetensors` needs to be installed explicitly too — `huggingface_hub`'s safetensors loader imports it lazily and fails with an unhelpful bare `NameError` if it's missing.

A second real finding: on CPU, the model's default `bfloat16` compute dtype measured **roughly 4x slower** than plain `float32` in our own timing (191s vs. 48s for an identical tiny workload) — bf16 matmul is only fast with hardware most CPUs lack. Both notebooks request `float32` explicitly when running on CPU, and check for a genuinely free GPU before requesting one (a GPU shared with other processes, e.g. a local LLM server, is common and worth checking for before assuming it's free):

```
def pick_device(min_free_gb=6.0):
    if not torch.cuda.is_available():
        return "cpu"
    free_bytes, _ = torch.cuda.mem_get_info()
    free_gb = free_bytes / 1e9
    return "cuda" if free_gb >= min_free_gb else "cpu"
```

1.3 Classification: UCI Adult Income

48,842 rows, mixed numeric/categorical, binary target (`<=50K` vs. `>50K`, imbalanced ~76/24). Both `TabFMClassifier` presets were run with a deliberately bounded context — TabFM's own model card states memory scales with context rows, so unlike the classical baselines (trained on the full ~39,073-row pool), TabFM was given a 2,000-row context:

```
clf_fast = TabFMClassifier(model=tabfm_model, n_estimators=8, batch_size=1, random_state=42)
clf_fast.fit(X_context, y_context.to_numpy())
proba_fast_all = clf_fast.predict_proba(X_eval_plus_new)
```

The `.ensemble()` preset configures the same frozen weights with a heavier recipe — feature crosses, SVD features, NNLS-weighted blending, and Platt/vector calibration:

```
clf_ens = TabFMClassifier.ensemble(tabfm_model, n_estimators=12, random_state=42)
```

Results (2,000-row context vs. classical baselines on the full 39,073-row training pool):

Model	Accuracy	ROC-AUC	Wall-clock
LogisticRegression	0.866	0.9154	full pool
XGBoost	0.880	0.9355	full pool
TabFM (fast)	0.878	0.9247	21.2s
TabFM (ensemble)	0.878	0.9231	146.5s (6.9x fast preset)

The ensemble preset cost 6.9x the wall-clock of the fast preset for a *worse* ROC-AUC — a real, measured example of "heavier isn't automatically better," not a hypothetical caveat.

A subgroup breakdown by `sex` and `race` (from the notebook's fairness-audit section, motivated by TabFM's own model card stating "performance on minority groups… is not fully characterised") on the 1,000-row evaluation set:

Subgroup	n	Base rate (>50K)	Accuracy	ROC-AUC
Female	345	0.113	0.951	0.974
Male	655	0.305	0.840	0.894
Amer-Indian-Eskimo	11	0.091	0.909	0.600
Asian-Pac-Islander	26	0.346	0.769	0.915
Black	99	0.131	0.939	0.972
White	851	0.251	0.872	0.920

Read the small-`n` rows with real caution — 11 and 13 rows respectively for the smallest subgroups is not enough to draw a confident conclusion, but the swing (ROC-AUC of 0.600 on Amer-Indian-Eskimo vs. 0.972 on Black) is large enough to flag as something you'd want a much larger evaluation set to actually verify before trusting this model on a genuinely high-stakes decision.

1.4 Regression: California Housing

20,640 rows, fully numeric, 8 features. A real data quirk confirmed directly in the notebook: the 1990 Census survey capped reported home values at $500,001, so 965 of 20,640 rows (4.7%) are pinned at exactly that ceiling — any model is structurally unable to predict genuinely higher values for those rows, since the training signal itself is truncated.

```
X_context, y_context = subsample(X_train_pool, y_train_pool, 2500)
reg = TabFMRegressor(model=tabfm_model, n_estimators=8, batch_size=1, random_state=42)
reg.fit(X_context, y_context.to_numpy())
```

Results (2,500-row context vs. full 16,512-row training pool for classical baselines):

Model	RMSE	R^2	Wall-clock
LinearRegression	0.7024	0.6108	full pool
XGBoost	0.4733	0.8232	full pool
TabFM (fast)	**0.4287**	**0.8550**	24.7s
TabFM (ensemble)	0.4298	0.8542	177.7s (7.2x fast preset)

This is the one head-to-head in this repository where a foundation model **beat full-data XGBoost outright**, using roughly 15% of the training pool as context. The ensemble preset again cost ~7x the wall-clock for a marginally worse result — the same pattern as the classification notebook.

Part 2 — The Regression Model Zoo (NYC Green Taxi)

Notebooks: `tabiclv2_regression_tutorial.ipynb`, `catboost_regression_tutorial.ipynb`, `lightgbm_regression_tutorial.ipynb`

2.1 The Dataset

581,835 real NYC Green Taxi trip records from December 2016, downloaded live via `sklearn.datasets.fetch_openml(data_`
The target is `tip_amount` in dollars. Three real data problems, confirmed and handled identically across all three notebooks:

- **Invalid negative tips** (refunds/corrections) — dropped.
- **Implausible trip durations** — parsed from `lpep_pickup_datetime`/`lpep_dropoff_datetime`, filtered to (0, 180) minutes.
- **A leakage trap**: `total_amount` is approximately fare + every surcharge *including the tip itself* — excluded from every model's features entirely.
- **Two genuinely high-cardinality categoricals**: `PULocationID` (233 distinct values) and `DOLocationID` (259 distinct values) — exactly the kind of column that breaks naive one-hot encoding and is where CatBoost's and LightGBM's native categorical handling earns its keep.
- **A dtype gotcha**: OpenML's loader auto-detects the low-cardinality numeric fee columns (`extra`, `mta_tax`, `improvement_surcharge`) as pandas `category` dtype despite being genuinely numeric — every notebook coerces them back explicitly:

```
for c in ["extra", "mta_tax", "improvement_surcharge", "passenger_count"]:
    df[c] = pd.to_numeric(df[c].astype(str), errors="coerce")
```

Cleaned to 578,293 rows; split 80/20 (`random_state=42`) into a ~462,634-row training pool and a shared 3,000-row evaluation sample used identically across all three notebooks.

2.2 TabICLv2

What it is: TabICLv2 comes from Inria's Soda team (Jingang Qu, David Holzmüller, Gaël Varoquaux, Marine Le Morvan) — `pip install tabicl`, scikit-learn compliant, **BSD-3-Clause licensed** (the most permissive license of any model in this repository; no commercial-use restriction). It shares the same row-compression idea TabFM later adopted, and was the first to introduce it, in the original ICML 2025 paper. Regression support (`TabICLRegressor`) is new in v2 — v1 and v1.1 were classification-only.

Documented scale: works well from 300 up to 100,000 training rows and up to 2,000 features; on an H100 GPU it fits and predicts a 50,000-row x 100-feature dataset in under 10 seconds, reported as 10x faster than TabPFN-2.5.

A real gotcha we hit: `huggingface_hub`'s accelerated `hf_xet` transfer protocol stalled indefinitely (0 bytes progress) fetching TabICLv2's ~109 MB checkpoint on our network, while a plain `curl` to the exact same URL worked, just slowly (~560 KB/s). Uninstalling the `hf_xet` package forces a plain HTTP fallback that completed in about 5 minutes:

```
uv pip uninstall --python {sys.executable} -y hf_xet
```

Measured scaling, on this machine's consumer GPU (not H100):

Context + query rows	n_estimators	Wall-clock
1,000	4	128.3s (CPU, before we found the GPU)
2,500	4–8	~7.6s (GPU)
20,000 + 3,000	8	**7.6s**

We locked in a **20,000-row context** for the real run — dramatically more headroom than Google TabFM could afford on the same hardware, a real, measured difference rather than a vendor claim.

```
reg = TabICLRegressor(device=DEVICE, n_estimators=8)
reg.fit(X_context, y_context.to_numpy())
pred_all = reg.predict(X_eval_plus_new)
```

Result: RMSE 2.5694, R^2 0.2613, in 7.6 seconds — beating the full-data linear baseline (RMSE 2.6538) despite using only 4.3% of the training pool.

2.3 CatBoost

What it is: Yandex's gradient-boosted tree library (2017, Apache-2.0, version 1.2.10 verified for this notebook). Two specific architectural choices set it apart from other GBM libraries:

1. **Symmetric (oblivious) decision trees** — the same splitting condition at every node of a given depth, across the entire tree. A full tree evaluates as a simple lookup table, giving faster inference and acting as built-in regularization.
2. **Ordered boosting + ordered target statistics** — rows are processed in a randomly permuted, causally-consistent order so a row's own target never leaks into the statistics used to predict it, specifically reducing overfitting from naive categorical target-encoding.

This means you hand CatBoost your raw categorical columns — however many distinct values they have — and it handles them internally, no one-hot/hashing/target-encoding required:

```
cat_idx = [X_train_pool.columns.get_loc(c) for c in CAT_COLS]
cb_model = CatBoostRegressor(iterations=500, learning_rate=0.1, depth=8,
                             loss_function="RMSE", random_state=42, verbose=False)
cb_model.fit(X_train_pool, y_train_pool, cat_features=cat_idx)
```

A real gotcha we hit: the same `extra`/`mta_tax`/`improvement_surcharge` dtype quirk from Section 2.1 raises a `CatBoostError` if a `category`-dtype column isn't declared in `cat_features` — CatBoost refuses to guess. Coercing those columns back to numeric first (already done in the shared prep) is what avoids this.

Result: trained on the full 462,634-row pool in **26.2 seconds**, RMSE 2.3301, R^2 0.3925 — clearly ahead of both the linear baseline and TabICLv2's bounded-context result, at the cost of using ~23x the training data.

2.4 LightGBM

What it is: Microsoft's gradient-boosted tree library (MIT license, version 4.6.0 verified for this notebook), built for speed and memory efficiency via two choices:

1. **Leaf-wise (best-first) growth** — always splits whichever single leaf reduces loss the most, regardless of depth, rather than growing every leaf at the current depth first. Reaches lower loss faster for the same leaf count, at some risk of overfitting if `num_leaves` isn't watched.
2. **Histogram-based split finding** — buckets continuous features into a fixed number of bins up front, trading a small amount of split precision for a large speed/memory win.

Native categorical support uses a Fisher-based partitioning algorithm on integer-encoded category codes, working directly on the pandas `category` dtype:

```
lgbm_model = lgb.LGBMRegressor(n_estimators=500, learning_rate=0.1, num_leaves=63, random_state=42)
lgbm_model.fit(X_train_pool, y_train_pool, categorical_feature=CAT_COLS)
```

Result: trained on the full 462,634-row pool in **1.9 seconds** — the fastest model in this entire repository, foundation models included — with the best RMSE (2.2608) and R^2 (0.4281) of the regression trio. Not a close-but-slower alternative to CatBoost; better *and* roughly 14x faster on this exact dataset and split.

2.5 Head-to-Head

Model	RMSE ↓	R^2 ↑	Wall-clock	Training data
LinearRegression	2.6538	0.2120	5.6–5.9s	Full pool
TabICLv2	2.5694	0.2613	**7.6s**	20,000-row context
CatBoost	2.3301	0.3925	26.2s	Full pool
LightGBM	**2.2608**	**0.4281**	**1.9s**	Full pool

LightGBM's win here is genuinely clean — best accuracy *and* fastest by a wide margin, training on the exact same data CatBoost used. TabICLv2's result is the more interesting one for anyone compute- or context-constrained: a real, honest win over the full-data linear baseline in a fraction of the time and data, even though it didn't catch the full-data GBMs on this particular task. See `COMPARISON.md` for the discussion of *why* the high-cardinality categoricals likely favor the full-data GBMs here.

Part 3 — The Classification Model Zoo (Forest Covertype)

Notebooks: `tabpfn_v3_classification_tutorial.ipynb`, `mitra_classification_tutorial.ipynb`, `xgboost_classification_tutorial.ipynb`

3.1 The Dataset

581,012 real cartographic records from the Roosevelt National Forest, Colorado, downloaded live via `sklearn.datasets.fetch_covtype()`. The task: predict which of 7 forest cover types grows on a 30x30m patch, from elevation, slope, distance-to-water/roads/fire-points, and soil/wilderness classification — no imagery. The target is meaningfully imbalanced, from 211,840 rows of the majority class down to 2,747 of the rarest (a 77x ratio).

scikit-learn ships wilderness area (4 values) and soil type (40 values) *already* one-hot encoded into 44 separate binary columns. Every notebook in this trio reconstructs them back into two compact categorical columns — a real, honest preprocessing step, not fabricated data, that both reduces dimensionality (54 → 12 columns) and gives every model raw categorical columns to actually demonstrate native categorical handling on:

```
wild_cols = [c for c in df.columns if c.startswith("Wilderness_Area_")]
soil_cols = [c for c in df.columns if c.startswith("Soil_Type_")]
df["Wilderness_Area"] = df[wild_cols].to_numpy().argmax(axis=1).astype(str)
df["Soil_Type"] = df[soil_cols].to_numpy().argmax(axis=1).astype(str)
```

Split 80/20, stratified (`random_state=42`), into a ~464,809-row training pool and a shared, stratified 3,000-row evaluation sample used identically across all three notebooks.

3.2 TabPFN v3

What it is: TabPFN ("Tabular Prior-Fitted Network") comes from Prior Labs (Hollmann, Müller, Purucker, Hutter, and collaborators), and is the model that originated the whole in-context tabular foundation model lineage this repository covers — its 2023 ICLR paper and 2025 Nature paper predate both TabICL and TabFM, which explicitly build on ideas TabPFN introduced.

The core idea, unchanged since the original paper: a transformer pretrained once on an enormous number of synthetic datasets sampled from a prior over plausible data-generating processes. At inference time it reads real training data as context and approximates Bayesian posterior inference over labels for new points, in one forward pass.

What's new in v3 (the current default in the `tabpfn` package) is scale:

Checkpoint	Recommended envelope (rows x features)
TabPFN-2.6 (previous default)	up to 100,000 x 2,000
TabPFN-3 (current default)	up to **1,000,000 x 200**, **100,000 x 2,000**, or **1,000 x 20,000**

Native categorical handling is built in. The real, documented constraint: *"TabPFN is slow to execute on a CPU… on CPU, only small datasets (1,000 samples) are feasible"* — GPU is close to a requirement, not a nice-to-have, for anything beyond toy-sized data.

```
from tabpfn import TabPFNClassifier

clf = TabPFNClassifier(device=DEVICE, random_state=42)
clf.fit(X_context, y_context.to_numpy())
proba_all = clf.predict_proba(X_eval_plus_new)
```

A real, current blocker, disclosed rather than worked around: as of this writing, the `tabpfn` package (v8.0.8) requires a **one-time, interactive license acceptance** before it will download model weights — a change from earlier TabPFN releases, and something that genuinely cannot be automated from a script or CI pipeline:

```
tabpfn.errors.TabPFNLicenseError: TabPFN requires a one-time license acceptance to download
model weights for local inference, but no interactive terminal is available.

To authenticate in a non-interactive environment:
  1. Open https://ux.priorlabs.ai in a browser and log in (or register)
  2. Accept the license on the Licenses tab
  3. Copy your API Key from https://ux.priorlabs.ai/account
  4. Set the environment variable: export TABPFN_TOKEN="<your-api-key>"
```

The notebook `tabpfn_v3_classification_tutorial.ipynb` in this repository is **complete and fully written** — theory, environment setup, EDA, the zero-shot classification call above, permutation importance, new-example inference, practical notes — it simply has not been executed end-to-end in this repository, because that one step requires a human with a browser, not an automatable pipeline. Anyone with a Prior Labs account can run it themselves in a few minutes by setting `TABPFN_TOKEN` before the first `.fit()` call.

3.3 Mitra

What it is: Mitra is AutoGluon's own tabular foundation model, built by the AutoGluon team at AWS and natively integrated into `TabularPredictor` — the same 3-line API AutoGluon uses for every other model it supports. Like TabPFN and TabICL, it's an in-context-learning transformer pretrained on synthetic data, but with a specific twist: a "principled pretraining approach by carefully selecting and mixing diverse synthetic priors to promote robust generalization" rather than one single synthetic-data generator. AutoGluon reports state-of-the-art results on TabRepo, TabZilla, AMLB, and TabArena, "especially excelling on small tabular datasets with fewer than 5,000 samples and 100 features."

Two things make Mitra distinctive in this repository: it's the only model here that supports **fine-tuning** in addition to zero-shot inference, and its weights are **fully open-source under Apache-2.0** — the only tabular foundation model in this entire project without a non-commercial-use restriction.

```
from autogluon.tabular import TabularPredictor

mitra_predictor = TabularPredictor(label="Cover_Type", path="./mitra_zeroshot_model")
mitra_predictor.fit(train_df, hyperparameters={"MITRA": {"fine_tune": False}})
proba_all = mitra_predictor.predict_proba(X_eval_plus_new)
```

Because Mitra's own documentation recommends staying under ~5,000 rows and ~100 features, it was given a **4,000-row context** — matching its documented sweet spot honestly, rather than penalizing it for a scale it was never built to handle zero-shot (the same principle applied to TabPFN v3's GPU-sized context in Section 3.2).

Uniquely among the foundation models in this repository, Mitra also supports fine-tuning — a bounded number of gradient steps on top of the pretrained weights:

```
mitra_predictor_ft = TabularPredictor(label="Cover_Type", path="./mitra_finetuned_model")
mitra_predictor_ft.fit(
    train_df,
    hyperparameters={"MITRA": {"fine_tune": True, "fine_tune_steps": 30}},
    time_limit=180,
)
```

Result (4,000-row context, same 3,000-row evaluation set as the rest of the trio):

Configuration	Accuracy	Balanced accuracy	Log loss	Wall-clock
Zero-shot	0.7347	0.3959	0.6398	9.4s
Fine-tuned (30 steps)	**0.8000**	**0.6094**	**0.4834**	108.5s (11.5x zero-shot)

Zero-shot, Mitra's raw accuracy (0.7347) barely edged the linear baseline (0.7317) — but its balanced accuracy (0.3959) was noticeably *worse* than the baseline's (0.5237), meaning it struggled disproportionately on the rarer cover types despite matching on overall accuracy. This is exactly the kind of gap a metric-only-accuracy comparison would miss. **Just 30 fine-tuning steps (108.5s) closed that gap and then some**, clearing the linear baseline on both accuracy and balanced accuracy — real, measured evidence that fine-tuning is worth the extra cost for this model, even though the fine-tuned result still falls well short of XGBoost trained on the full dataset (Section 3.4) — an expected outcome given Mitra was deliberately kept inside its documented small-data envelope rather than given the full 464,809-row pool.

3.4 XGBoost

What it is: the gradient-boosted tree library that set the tabular baseline for over a decade (Apache-2.0). Unlike every foundation model in this repository, XGBoost trains a fresh ensemble of trees directly on your data, using second-order (Newton) gradient information plus explicit L1/L2 regularization on leaf weights baked into its objective. Modern XGBoost (v2.x+) supports native categorical features via `enable_categorical=True` and a histogram-based training path.

The Covertype notebook makes a point of comparing an untuned configuration against a genuinely tuned one, to test the claim that "a fully-tuned XGBoost remains the undisputed king" rather than just running defaults:

```
xgb_default = xgb.XGBClassifier(n_estimators=200, max_depth=6, learning_rate=0.3,
                                tree_method="hist", enable_categorical=True,
                                objective="multi:softprob", num_class=7, random_state=42)
xgb_default.fit(X_train_enc, y_train_pool_xgb)

search = RandomizedSearchCV(
    xgb.XGBClassifier(tree_method="hist", enable_categorical=True,
                      objective="multi:softprob", num_class=7, random_state=42),
    param_distributions={
        "n_estimators": [200, 400, 600], "max_depth": [6, 8, 10],
        "learning_rate": [0.05, 0.1, 0.2], "subsample": [0.7, 0.9, 1.0],
        "colsample_bytree": [0.7, 0.9, 1.0],
    },
```

```
    n_iter=8, cv=3, scoring="accuracy", random_state=42, n_jobs=-1,
)
search.fit(X_tune, y_tune)  # 30,000-row subsample, tuning would be too slow on the full pool
```

A real gotcha handled explicitly: XGBoost's multiclass objective expects zero-indexed labels; `Cover_Type` ships as 1–7. Every notebook in this trio shifts labels by exactly 1, in one place, right after the split, to avoid an off-by-one bug propagating downstream.

Result:

Configuration	Accuracy	Balanced accuracy	F1 (macro)	Wall-clock
LogisticRegression (baseline)	0.7317	0.5237	0.5428	29.2s
XGBoost (default-ish)	0.9190	0.9248	0.9263	247.3s
XGBoost (tuned)	**0.9767**	**0.9641**	**0.9605**	71.2s + 55.6s tuning

An 8-configuration, 3-fold randomized search, run on a 30,000-row subsample in **55.6 seconds**, found hyperparameters (`max_depth=10`, `n_estimators=400`, `learning_rate=0.2`) that took the full-data refit from 91.9% to 97.7% accuracy — the single largest accuracy jump measured anywhere in this repository, and it came from tuning effort, not from switching model families.

3.5 Head-to-Head

Model	Accuracy ↑	Balanced accuracy ↑	Log loss ↓	Wall-clock	Training data
LogisticRegression	0.7317	0.5237	0.6285	29.2s	Full pool
Mitra (zero-shot)	0.7347	0.3959	0.6398	9.4s	4,000-row context
Mitra (fine-tuned)	0.8000	0.6094	0.4834	108.5s	4,000-row context
XGBoost (default)	0.9190	0.9248	0.2171	247.3s	Full pool
XGBoost (tuned)	**0.9767**	**0.9641**	**0.0667**	71.2s + 55.6s tuning	Full pool
TabPFN v3	*not executed — see Section 3.2*				

Read this table alongside Section 2.5's regression head-to-head: the pattern repeats. A classical GBM given the full dataset and a real (if light) tuning pass wins outright; a foundation model kept honestly within its documented small-context envelope is competitive with an untuned linear baseline and improves substantially with fine-tuning, but doesn't close the gap to a tuned, full-data GBM on a dataset this large. Neither result contradicts the other — they're both true, at different points on the data-scale and compute-budget spectrum. See Part 4 for the full synthesis.

Part 4 — Cross-Cutting Lessons

Eight notebooks, six models, two large real datasets, one consistent methodology (fixed random seeds, a shared held-out evaluation set within each trio, classical baselines given the full dataset with no artificial handicap). A few things held up across every single comparison in this repository, not just one lucky result:

1. A tuned classical GBM, given the full dataset, is still the strongest single result here. 97.7% accuracy on 7-class Covertype, the best RMSE on NYC Taxi tip prediction, and the fastest training time in both trios. Nothing in this repository overturns a decade of "well-tuned XGBoost/ LightGBM/CatBoost is a very strong tabular baseline" — if anything, it reinforces it.

2. Foundation models are genuinely competitive on a first pass, with zero tuning, using a fraction of the data — and sometimes they win outright. Google TabFM beat full-data XGBoost on California Housing. TabICLv2 beat a full-data linear baseline on NYC Taxi tips using 4.3% of the training pool, in 7.6 seconds. Neither of these required a single hyperparameter search.

3. Heavier "ensemble" presets on foundation models did not pay for themselves, either time we tested it. Both of Google TabFM's `.ensemble()` runs cost 7-9x the wall-clock of the fast preset for a *worse* result on the held-out set. If you're deciding whether to spend the extra compute, measure it — don't assume it.

4. Tuning effort is not optional if you want a fair comparison, and its effect can dwarf the choice of model family. The single largest accuracy swing in this entire repository — 91.9% to 97.7% — came from a 55-second randomized search on a classical GBM, not from switching to a foundation model.

5. Context-size and licensing constraints are real, model-specific, and worth checking before you start a project, not after. TabPFN v3 needed a manual license step that blocked full automation. Mitra's documentation steered us toward a 4,000-row context. TabICLv2 and Google TabFM each have their own hard limits (10 classes for TabFM; a documented 100,000-row/2,000-feature envelope for TabICLv2). None of this is a flaw in any specific model — it's the actual, current state of the tooling, and worth knowing before you commit to one for a real project.

6. Real datasets have real problems, and every notebook in this repository handles them explicitly rather than hiding them. Negative tip amounts, a target-leakage column, one-hot columns that needed reconstructing back into genuine categoricals, a dtype quirk that silently misclassifies numeric fee columns as categorical, an off-by-one label-indexing requirement — these are the actual engineering work of a tabular ML project, and they showed up in every single notebook, not just the "interesting" ones.

References

Google TabFM - Google Research blog: "Introducing TabFM" - github.com/google-research/tabfm - huggingface.co/google/tabfm-1.0.0-pytorch

TabICLv2 - github.com/soda-inria/tabicl - TabICLv2 paper (arXiv:2602.11139) - TabICLv1 / ICML 2025 paper (arXiv:2502.05564)

TabPFN v3 - github.com/priorlabs/tabpfn - TabPFN-2.5 technical report (arXiv:2511.08667) - Original Nature paper

Mitra - huggingface.co/autogluon/mitra-classifier - github.com/autogluon/autogluon - Official AutoGluon foundational-models tutorial

Classical GBMs - github.com/dmlc/xgboost - github.com/catboost/catboost / catboost.ai/docs - github.com/microsoft/LightGBM / lightgbm.readthedocs.io

Datasets - UCI Adult / Census Income (OpenML) - California Housing dataset (scikit-learn docs) - NYC TLC Trip Record Data / dataset on OpenML (id 42208) - Covertype dataset (UCI / scikit-learn)

Benchmarks referenced - TabArena benchmark (arXiv:2506.16791)